EXVUL

SMART CONTRACT AUDIT REPORT

HolmesAI Smart Contract

DECEMBER 2025

Contents

1. EXECUTIVE SUMMARY

ExVul Web3 Security was engaged by **HolmesAI** to review smart contract implementation. The assessment was conducted in accordance with our systematic approach to evaluate potential security issues based upon customer requirement. The report provides detailed recommendations to resolve the issue and provide additional suggestions or recommendations for improvement.

The outcome of the assessment outlined in chapter 3 provides the system's owners a full description of the vulnerabilities identified, the associated risk rating for each vulnerability, and detailed recommendations that will resolve the underlying technical issue.

1.1 Methodology

To standardize the evaluation, we define the following terminology based on OWASP Risk Rating Methodology [10] which is the gold standard in risk assessment using the following risk models:

- **Likelihood**: represents how likely a particular vulnerability is to be uncovered and exploited in the wild.
- **Impact**: measures the technical loss and business damage of a successful attack.
- **Severity**: determine the overall criticality of the risk.

Likelihood can be: High, Medium and Low and impact are categorized into: High, Medium, Low, Informational. Severity is determined by likelihood and impact and can be classified into five categories accordingly: Critical, High, Medium, Low, Informational shown in table 1.1.

Likelihood \ IMPACT	Informational	Low	Medium	High
High	INFO	MEDIUM	HIGH	CRITICAL
Medium	INFO	LOW	MEDIUM	HIGH
Low	INFO	LOW	LOW	MEDIUM

Table 1.1 Overall Risk Severity

To evaluate the risk, we will be going through a list of items, and each would be labelled with a severity category. The audit was performed with a systematic approach guided by a comprehensive assessment list carefully designed to identify known and impactful security issues. If our tool or analysis does not identify any issue, the contract can be considered safe regarding the assessed item. For any discovered issue, we might further deploy contracts on our private test environment and run tests to confirm the findings. If necessary, we would additionally build a PoC to demonstrate the possibility of exploitation. The concrete list of check items is shown in Table 1.2.

- **Basic Coding Bugs**: We first statically analyze given smart contracts with our proprietary static code analyzer for known coding bugs, and then manually verify (reject or confirm) all the issues found by our tool.
- **Code and business security testing**: We further review business logics, examine system operations, and place DeFi-related aspects under scrutiny to uncover possible pitfalls and/or bugs.
- **Additional Recommendations**: We also provide additional suggestions regarding the coding and development of smart contracts from the perspective of proven programming practices.

Category	Assessment Item
Basic Coding Assessment	• Apply Verification Control • Authorization Access Control • Forged Transfer Vulnerability • Forged Transfer Notification • Numeric Overflow • Transaction Rollback Attack • Transaction Block Stuffing Attack • Soft Fail Attack • Hard Fail Attack • Abnormal Memo • Abnormal Resource Consumption • Secure Random Number

Advanced Source Code Scrutiny	• Asset Security • Cryptography Security • Business Logic Review • Source Code Functional Verification • Account Authorization Control • Sensitive Information Disclosure • Circuit Breaker • Blacklist Control • System API Call Analysis • Contract Deployment Consistency Check • Abnormal Resource Consumption
Additional Recommendations	• Semantic Consistency Checks • Following Other Best Practices

Table 1.2: The Full List of Assessment Items

To better describe each issue we identified, we categorize the findings with Common Weakness Enumeration (CWE-699) [14], which is a community-developed list of software weakness types to better delineate and organize weaknesses around concepts frequently encountered in software development.

2. FINDINGS OVERVIEW

2.1 Project Info And Contract Address

Project Name	Audit Time	Language
HolmesAI	24/12/2025 - 12/01/2026	Solidity

Repository
Airdrop.sol
HOL.sol
MultiSigWalletDeploy.sol
Staking.sol
StakingUSD.sol
Vesting.sol
VestingFactory.sol

Commit Hash
3a6cb5c10c05688fbcb22c1f6d460044
966cf0203142bb05024a36f28cdb93ed
2356c8fca0787d57e5415493c74d42a4
079431aa7dd8870c9b8e09899105194a
9907c3814f6539bc24edcb4c21d39a21
4ec7c3af569e00cd71b036e69895b1b0
4a3ea90e6a3e8f4881fb59740962c00e

2.2 Summary

Severity	Found
CRITICAL	2
HIGH	4
MEDIUM	4
LOW	5
INFO	3

2.3 Key Findings

Severity	Findings Title	Status
CRITICAL	Unauthorized Airdrop Claiming Enables Permanent Fund Confiscation	Fixed
CRITICAL	Lock Duration Unit Mismatch Causes Part One Claiming DoS	Fixed
HIGH	Missing Token Approval Breaks Staking Path	Fixed
HIGH	Exponential State Corruption in Part Two Claiming	Fixed
HIGH	Deviation from Economic Model due to Precision Loss in previewStake	Fixed
HIGH	The ownership verification of the pledge certificate is missing	Fixed
MEDIUM	Incorrect call order caused the state to fail to update	Fixed
MEDIUM	previewStake() has a design and implementation inconsistency	Fixed
MEDIUM	State Corruption via Reentrancy in stake	Fixed
MEDIUM	Logic Error in unstake(0) leads to Valid Stake Deletion	Fixed
LOW	Missing Boundary Checks for start	Acknowledge
LOW	Missing check for unlockInterval	Fixed
LOW	amount of 0 leads to a waste of resources	Fixed
LOW	Incorrect Share Minting Allows Fee-On-Transfer Tokens to Drain Vault Funds	Fixed
LOW	Reset Min/MaxLockDuration and add limits	Fixed
INFO	Inconsistency between code and comments	Fixed
INFO	Duplicate checks in update cause redundant traversal	Fixed

Severity	Findings Title	Status
INFO	No explicit existence check was performed on the non-existent stakeId	Fixed

Table 2.3: Key Audit Findings

3. DETAILED DESCRIPTION OF FINDINGS

3.1 Unauthorized Airdrop Claiming Enables Permanent Fund Confiscation

SEVERITY: CRITICAL **STATUS:** Fixed

PATH:

`Airdrop.sol`

DESCRIPTION:

claimPartOne accepts claimer as a parameter and validates the Merkle proof against this address, but fails to verify that msg.sender matches the claimer.

```
//  Airdrop.sol
    function claimPartOne(
        uint256 partOneLockDuration,
        address claimer,
        uint256 totalAllocated,
        bytes32[] calldata merkleProof
    ) public {
        // ...

        bytes32 leaf;
        assembly {
            mstore(0x00, claimer)
            mstore(0x20, totalAllocated)
            let innerHash := keccak256(0x00, 0x40)
            mstore(0x00, innerHash)
            leaf := keccak256(0x00, 0x20)
        }
        require(
            MerkleProof.verify(merkleProof, MERKLE_ROOT, leaf),
            "Airdrop: invalid merkle proof"
        );

        uint256 partOneAmount = Math.mulDiv(
            totalAllocated,
            PART_ONE_PERCENT,
```

```
            100
        );
        uint256 partTwoAmount = totalAllocated - partOneAmount;
        totalClaimed += partOneAmount;

        if (partOneLockDuration == 0) {
            partTwo.start = block.timestamp;
            partTwo.end = block.timestamp;
            recoverableHOL += partTwoAmount;
            HOL.safeTransfer(claimer, partOneAmount);
        } else {
            uint256 partTwoDuration = calculatePartTwoDuration(
                partOneLockDuration
            );
            partTwo.start = block.timestamp;
            partTwo.end = block.timestamp + partTwoDuration;
            partTwo.total = partTwoAmount;
            STAKING.stake(partOneAmount, partOneLockDuration, claimer);
        }

        // ...
    }
```

IMPACT:

Attacker calls claimPartOne(0, victimAddress, amount, proof) with partOneLockDuration = 0. if (partOneLockDuration == 0) { partTwo.start = block.timestamp; partTwo.end = block.timestamp; // Part Two immediately expires recoverableHOL += partTwoAmount; // 75% marked for treasury HOL.safeTransfer(claimer, partOneAmount); // Only 25% to victim } Since partTwo.total is never set when lockDuration == 0, the victim cannot claim Part Two via claimPartTwo.

RECOMMENDATIONS:

Implement caller authorization by enforcing that only the beneficiary can claim their own allocation.

```
    function claimPartOne(
        uint256 partOneLockDuration,
        address claimer,
        uint256 totalAllocated,
        bytes32[] calldata merkleProof
```

```
    ) public {
+       require(msg.sender == claimer, "Airdrop: caller is not the
    claimer");
        PartTwo storage partTwo = claimerToPartTwo[claimer];
        require(partTwo.end == 0, "claimer already claimed part one");
        require(
            partOneLockDuration == 0 ||
                partOneLockDuration >= PART_ONE_MIN_LOCK_DURATION,
            "Airdrop: part one lock duration must be >=
    PART_ONE_MIN_LOCK_DURATION or 0"
        );
        // ...
    }
```

3.2 Lock Duration Unit Mismatch Causes Part One Claiming DoS

SEVERITY: CRITICAL **STATUS:** Fixed

PATH:

`Airdrop.sol, Staking.sol`

DESCRIPTION:

Airdrop defines lock duration constants in seconds (e.g., 30 days), while Staking expects the lockDuration argument to be in days.

```
// Airdrop.sol
    function claimPartOne(
        uint256 partOneLockDuration,
        address claimer,
        uint256 totalAllocated,
        bytes32[] calldata merkleProof
    ) public {
        PartTwo storage partTwo = claimerToPartTwo[claimer];
        require(partTwo.end == 0, ''claimer already claimed part one'');
        require(
            partOneLockDuration == 0 ||
                partOneLockDuration >= PART_ONE_MIN_LOCK_DURATION,
            ''Airdrop: part one lock duration must be >=
   PART_ONE_MIN_LOCK_DURATION or 0''
        );
        require(
            partOneLockDuration <= PART_ONE_MAX_LOCK_DURATION,
            ''Airdrop: part one lock duration must be <=
   PART_ONE_MAX_LOCK_DURATION''
        );

        // ...

        if (partOneLockDuration == 0) {
            partTwo.start = block.timestamp;
            partTwo.end = block.timestamp;
            recoverableHOL += partTwoAmount;
```

```
            HOL.safeTransfer(claimer, partOneAmount);
        } else {
            uint256 partTwoDuration = calculatePartTwoDuration(
                partOneLockDuration
            );
            partTwo.start = block.timestamp;
            partTwo.end = block.timestamp + partTwoDuration;
            partTwo.total = partTwoAmount;
            STAKING.stake(partOneAmount, partOneLockDuration, claimer);
        }

        emit ClaimPartOne(
            claimer,
            totalAllocated,
            partOneLockDuration,
            partOneAmount
        );
    }
// Staking.sol
    function initialize(address initialOwner) public initializer {
        __ERC1155_init('');
        __Ownable_init(initialOwner);

        lockConversionRatio = 2;
        maxLockDuration = 2 * 365; // 2 years
        minLockDuration = 6; // 6 days

        /// @dev unstake certificate token id is 1 ... n, NFT
        _nextStakeId = 1;
    }

    function stake(
        uint256 amount,
        uint256 lockDuration,
        address receiver
    ) public {
        require(amount > 0, ''amount must > 0'');
        require(
            lockDuration >= minLockDuration || lockDuration == 0,
            ''lock duration must >= min lock duration or == 0''
        );
        require(
            lockDuration <= maxLockDuration,
```

```
            "lock duration must <= max lock duration"
        );

        /// @dev calculate the stake at and lock until
        uint256 stakeAt = block.timestamp;
        uint256 lockUntil = stakeAt + lockDuration * 1 days;

        // ...
    }
```

IMPACT:

When a user calls Airdrop.claimPartOne with a valid lock duration (e.g., 30 days = 2,592,000), this large integer value is passed directly to Staking.stake. The Staking contract interprets this input as 2,592,000 days. This triggers the check require(lockDuration <= maxLockDuration) (where maxLockDuration is 730), causing the transaction to revert immediately.

RECOMMENDATIONS:

Use days integer units for Airdrop configuration to match Staking expectations.

```
// Airdrop.sol

-   uint256 public constant PART_ONE_MIN_LOCK_DURATION = 30 days;
+   uint256 public constant PART_ONE_MIN_LOCK_DURATION = 30; // 30 days
-   uint256 public constant PART_ONE_MAX_LOCK_DURATION = 2 * 365 days;
+   uint256 public constant PART_ONE_MAX_LOCK_DURATION = 2 * 365; // 2
    years

    function calculatePartTwoDuration(
        uint256 partOneLockDuration
    ) public pure returns (uint256 partTwoDuration) {
        partTwoDuration =
            PART_TWO_MAX_DURATION -
            Math.mulDiv(
                PART_TWO_MAX_DURATION - PART_TWO_MIN_DURATION,
-               partOneLockDuration - PART_ONE_MIN_LOCK_DURATION,
-               PART_ONE_MAX_LOCK_DURATION - PART_ONE_MIN_LOCK_DURATION
+               (partOneLockDuration * 1 days) -
    (PART_ONE_MIN_LOCK_DURATION * 1 days),
+               (PART_ONE_MAX_LOCK_DURATION * 1 days) -
```

```
    (PART_ONE_MIN_LOCK_DURATION * 1 days)
            );
        partTwoDuration = Math.max(
            Math.min(partTwoDuration, PART_TWO_MAX_DURATION),
            PART_TWO_MIN_DURATION
        );
    }
```

3.3 Missing Token Approval Breaks Staking Path

SEVERITY: HIGH **STATUS:** Fixed

PATH:

`Airdrop.sol, Staking.sol`

DESCRIPTION:

Airdrop.claimPartOne calls STAKING.stake() when users choose to lock their tokens, but the Airdrop contract never approves the Staking contract to transfer HOL tokens on its behalf.

```
// Airdrop.sol
    function claimPartOne(
        uint256 partOneLockDuration,
        address claimer,
        uint256 totalAllocated,
        bytes32[] calldata merkleProof
    ) public {
        // ...

        if (partOneLockDuration == 0) {
            partTwo.start = block.timestamp;
            partTwo.end = block.timestamp;
            recoverableHOL += partTwoAmount;
            HOL.safeTransfer(claimer, partOneAmount);
        } else {
            uint256 partTwoDuration = calculatePartTwoDuration(
                partOneLockDuration
            );
            partTwo.start = block.timestamp;
            partTwo.end = block.timestamp + partTwoDuration;
            partTwo.total = partTwoAmount;
            STAKING.stake(partOneAmount, partOneLockDuration, claimer);
   //@audit Calls external contract
        }
    }
// Staking.sol
function stake(uint256 amount, uint256 lockDuration, address receiver)
```

```
    public {
     // ...
     HOL.safeTransferFrom(msg.sender, address(this), amount); //
    msg.sender = Airdrop
     // ...
}
function initialize(address initialOwner) public initializer {
    __Ownable_init(initialOwner);
    __UUPSUpgradeable_init();
}
```

IMPACT:

All claimPartOne calls with partOneLockDuration > 0 will revert with “ERC20: insufficient allowance”. Claimers attempting to stake are forced to either: - Choose lockDuration = 0 and forfeit 75% of their allocation (Part Two) - Be unable to claim their airdrop at all

RECOMMENDATIONS:

Add token approval to initialization.

```
    function initialize(address initialOwner) public initializer {
        __Ownable_init(initialOwner);
        __UUPSUpgradeable_init();
+       HOL.forceApprove(address(STAKING), type(uint256).max);
    }
```

3.4 Exponential State Corruption in Part Two Claiming

SEVERITY: HIGH **STATUS:** Fixed

PATH:

`Airdrop.sol`

DESCRIPTION:

claimPartTwo incorrectly handles the accounting of claimed tokens. Add the cumulative released amount partTwoReleased to the cumulative claimed amount partTwo.claimed, instead of updating partTwo.claimed to match the current released total or adding only the delta.

```
// Airdrop.sol
    function claimPartTwo(address claimer) public {
        PartTwo storage partTwo = claimerToPartTwo[claimer];
        require(partTwo.total > partTwo.claimed, 'no part two to claim');

        uint256 partTwoReleased = calculatePartTwoReleased(
            partTwo.start,
            partTwo.end,
            block.timestamp,
            partTwo.total
        );
        uint256 partTwoClaimable = partTwoReleased - partTwo.claimed;
        partTwo.claimed += partTwoReleased;
        totalClaimed += partTwoClaimable;
        HOL.safeTransfer(claimer, partTwoClaimable);
        // ...
    }
```

IMPACT:

partTwo.claimed grows exponentially faster than intended effectively double counting historical releases with every claim. Scenario: - Week 1: Released 10. claimed becomes 0 + 10 = 10. User gets 10. (OK) - Week 2: Released 20. claimed becomes 10 + 20 = 30. (Should be 20). - Week 3: Released 30. partTwoClaimable = 30 - 30 = 0. User gets 0. Users stop receiving tokens much earlier than intended

as shown in Week 3 above. If partTwo.claimed rapidly exceeds partTwo.total, permanently blocking future claims.

RECOMMENDATIONS:

Update partTwo.claimed to equal the current cumulative released amount.

```
    function claimPartTwo(address claimer) public {
        PartTwo storage partTwo = claimerToPartTwo[claimer];
        require(partTwo.total > partTwo.claimed, "no part two to claim");

        uint256 partTwoRereleased = calculatePartTwoReleased(
            partTwo.start,
            partTwo.end,
            block.timestamp,
            partTwo.total
        );
        uint256 partTwoClaimable = partTwoReleased - partTwo.claimed;
-       partTwo.claimed += partTwoReleased;
+       partTwo.claimed = partTwoReleased;
        totalClaimed += partTwoClaimable;
        HOL.safeTransfer(claimer, partTwoClaimable);
        // ...
    }
```

3.5 Deviation from Economic Model due to Precision Loss in previewStake

SEVERITY: HIGH **STATUS:** Fixed

PATH:

`Staking.sol`

DESCRIPTION:

previewStake calculates the sHOL reward using integer division on the multiplier factor before applying it to the principal amount.

```
// Staking.sol
    function previewStake(
        uint256 amount,
        uint256 lockDuration
    ) public view returns (uint256) {
        return amount * Math.max(1, Math.mulDiv(
            (1 + lockConversionRatio), // 3
            (lockDuration - minLockDuration), // duration - 6
            maxLockDuration // 730
        ));
    }
```

IMPACT:

The Documentation states: “If locked, users will get extra sHOL… proportionate to lock duration”. However, Math.mulDiv performs (3 * (duration - 6)) / 730. Due to integer truncation: 1. For duration < 249: Result is 0. Max(1, 0) is 1. Impact: No bonus. 2. For 249 <= duration < 493: Result is 1. Max(1, 1) is 1. Impact: No bonus. 3. Only for duration >= 493: Result >= 2. Bonus applies. where users locking for anything less than 1.3 years (493 days) receive zero additional sHOL, exactly the same as users locking for the minimum duration.

RECOMMENDATIONS:

Perform multiplication by amount inside the mulDiv function to maintain precision.

```
    function previewStake(
        uint256 amount,
        uint256 lockDuration
    ) public view returns (uint256) {
-       return amount * Math.max(1, Math.mulDiv(
-           (1 + lockConversionRatio),
-           (lockDuration - minLockDuration),
-           maxLockDuration
-       ));

+       uint256 baseAmount = amount;
+       if (lockDuration == 0) return baseAmount;
+
+       // Calculate linear bonus
+       // amount * ratio * duration / max
+       uint256 bonus = Math.mulDiv(
+           amount * lockConversionRatio,
+           lockDuration,
+           maxLockDuration
+       );
+
+       return baseAmount + bonus;
    }
```

3.6 The ownership verification of the pledge certificate is missing

SEVERITY: HIGH **STATUS:** Fixed

PATH:

`Staking.sol`

DESCRIPTION:

In _unstake(), the contract destroys the NFT corresponding to the passed-in stakeId, but it does not check whether the caller owns the NFT. This allows an attacker to prevent others from unstaking the NFT by destroying someone else's stakeInfo.

```
// Staking.sol
    function _unstake(
        uint256 stakeId,
        uint256 index,
        address receiver
    ) internal {
        StakeInfo memory stakeInfo = stakeIdToStakeInfo[stakeId];
        require(
            stakeInfo.lockUntil <= block.timestamp,
            "stake is not unlocked"
        );

        /// @dev Burn the sHOL
        _burn(msg.sender, stakeInfo.sHOL);

        /// @dev Emit burn event for staking certificate NFT
        stakingCertContract.emitBurnEvent(msg.sender, stakeId);

        /// @dev clean the stake info and stake list
        _removeStakeInfoFromStaker(msg.sender, stakeId, index);
        delete stakeIdToStakeInfo[stakeId];

        /// @dev Transfer the HOL to the receiver
        HOL.safeTransfer(receiver, stakeInfo.HOL);

        emit Unstaked(
```

```
            msg.sender,
            receiver,
            stakeId,
            stakeInfo.HOL,
            stakeInfo.sHOL
        );
    }
```

IMPACT:

Destroying someone else's stakeInfo prevents legitimate users from unstaking, resulting in financial losses.

RECOMMENDATIONS:

Check if stakeId belongs to msg.sender.

```
    function _unstake(
        uint256 stakeId,
        uint256 index,
        address receiver
    ) internal {
        StakeInfo memory stakeInfo = stakeIdToStakeInfo[stakeId];

+        require(
+            stakeInfo.staker == msg.sender,
+            "Caller is not the stake owner"
+        );

        require(
            stakeInfo.lockUntil <= block.timestamp,
            "stake is not unlocked"
        );

        // ...
    }
```

3.7 Incorrect call order caused the state to fail to update

SEVERITY: MEDIUM **STATUS:** Fixed

PATH:

StakingUSD.sol

DESCRIPTION:

unstake() allows users with record.staked set to 0 to call it. When the user passes amount = 0 as an argument, the transaction can be executed normally, thus updating record.updateTime. The problem is that when the user calls stake(), walletToHolmesAI and holmesAIToWallet records will not be created.

```
// StakingUSD.sol
function unstake(string calldata holmesAI, uint256 amount) external {
    StakeRecord storage record = stakes[msg.sender][holmesAI];

    record.credit += calculateCredit(
        record.staked,
        block.timestamp - record.updateTime
    );
    record.staked -= amount;
    record.updateTime = block.timestamp;

    _burn(msg.sender, amount);
    USD.safeTransfer(msg.sender, amount);

    emit Unstake(msg.sender, holmesAI, amount, record.credit);
}
function stake(string calldata holmesAI, uint256 amount) external {
    StakeRecord storage record = stakes[msg.sender][holmesAI];

    if (record.updateTime == 0) {
        walletToHolmesAI[msg.sender].push(holmesAI);
        holmesAIToWallet[holmesAI].push(msg.sender);
    }

    // ...
```

```
}
```

IMPACT:

If unstake becomes stake, it will result in no record of the user's stake, affecting data collection.

RECOMMENDATIONS:

When calling unstake, check that record.staked and amount are not 0.

```
function unstake(string calldata holmesAI, uint256 amount) external {
+   require(amount > 0, 'Invalid unstake amount');
    StakeRecord storage record = stakes[msg.sender][holmesAI];

+   require(record.staked >= amount, 'Invalid staked amount');

    record.credit += calculateCredit(
        record.staked,
        block.timestamp - record.updateTime
    );
    record.staked -= amount;
    record.updateTime = block.timestamp;

    _burn(msg.sender, amount);
    USD.safeTransfer(msg.sender, amount);

    emit Unstake(msg.sender, holmesAI, amount, record.credit);
}
```

3.8 previewStake() has a design and implementation inconsistency

SEVERITY: MEDIUM **STATUS:** Fixed

PATH:

`Staking.sol`

DESCRIPTION:

previewStake() will calculate the number of sHOL tokens given to the user based on the input number of HOL tokens. 1. The documentation clearly states that locking for 2 years will grant triple the reward, but this cannot be achieved due to the calculation of lockDuration - minLockDuration. 2. Also due to the difference between lockDuration and minLockDuration, users do not receive any extra reward after locking for the minimum time. 3. When a user does not want to set a lock period and passes in lockDuration = 0, lockDuration - minLockDuration will trigger a revert, and the transaction will fail.

```
// Staking.sol
function previewStake(
    uint256 amount,
    uint256 lockDuration
) public view returns (uint256) {
    return amount * Math.max(1, Math.mulDiv(
        (1 + lockConversionRatio),
        (lockDuration - minLockDuration),
        maxLockDuration
    ));
}
function stake(
    uint256 amount,
    uint256 lockDuration,
    address receiver
) public {
    require(amount > 0, "amount must > 0");
    require(
        lockDuration >= minLockDuration || lockDuration == 0,
        "lock duration must >= min lock duration or == 0"
    );
    require(
        lockDuration <= maxLockDuration,
```

```
        "lock duration must <= max lock duration"
    );

    // ...

    /// @dev Store the stake info
    uint256 sHOL = previewStake(amount, lockDuration);

    // ...

    emit Staked(msg.sender, receiver, stakeId, amount, sHOL, lockUntil);
}
```

IMPACT:

If the economic model design fails to meet expectations, it will harm users' interests and affect their user experience.

RECOMMENDATIONS:

Remove lockDuration - minLockDuration and use lockDuration directly.

```
function previewStake(
    uint256 amount,
    uint256 lockDuration
) public view returns (uint256) {
    return amount * Math.max(1, Math.mulDiv(
        (1 + lockConversionRatio),
-       (lockDuration - minLockDuration),
+       lockDuration,
        maxLockDuration
    ));
}
```

3.9 State Corruption via Reentrancy in stake

SEVERITY: MEDIUM **STATUS:** Fixed

PATH:

`Staking.sol`

DESCRIPTION:

stake executes an external call mintBatch before updating the stakerToStakeIds mapping. allows a malicious actor to re-enter the contract via the onERC1155BatchReceived callback.

```
// Staking.sol
    function stake(
        uint256 amount,
        uint256 lockDuration,
        address receiver
    ) public {
        // ...

        uint256 stakeId = _nextStakeId++;

        /// @dev Store the stake info
        uint256 sHOL = previewStake(amount, lockDuration);
        stakeIdToStakeInfo[stakeId] = StakeInfo(
            amount,
            sHOL,
            stakeAt,
            lockUntil
        );

        /// @dev Mint the sHOL and unstake certificate
        uint256[] memory tokenIds = new uint256[](2);
        tokenIds[0] = sHOL_TOKEN_ID;
        tokenIds[1] = stakeId;
        uint256[] memory amounts = new uint256[](2);
        amounts[0] = sHOL;
        amounts[1] = 1;
        _mintBatch(receiver, tokenIds, amounts, '');    //@audit External
```

```
    call before state update

        /// @dev Store the staker to stake ids
        stakerToStakeIds[receiver].push(stakeId);
        stakeIdToIndex[stakeId] = stakerToStakeIds[receiver].length - 1;
      //@audit State updates happen after external call

        // ...
    }
```

IMPACT:

An attacker can corrupt their stake list by following these steps: 1. User has an existing stake (Stake A) at index 0. 2. User calls stake(...) to create Stake B. 3. Inside mintBatch, the attacker's onERC1155BatchReceived callback is triggered. 4. Attacker calls unstake(StakeB). 5. Since stakeB hasn't been pushed to stakerToStakeIds yet, stakeIdToIndex[StakeB] returns default 0. 6. unstake incorrectly removes the stake at index 0 (Stake A). 7. The original stake function resumes and pushes Stake B to the list. Result: - Stake A (Index 0): Removed from stakerToStakeIds but still exists in stakeIdToStakeInfo. User loses access to this stake (cannot unstake normally). - Stake B: Exists in stakerToStakeIds but has been burned/deleted in stakeIdToStakeInfo. - State is permanently corrupted by the user.

RECOMMENDATIONS:

Moving state updates before external calls.

```
    function stake(
        uint256 amount,
        uint256 lockDuration,
        address receiver
    ) public {
        // ...

        /// @dev Store the stake info
        uint256 sHOL = previewStake(amount, lockDuration);
        stakeIdToStakeInfo[stakeId] = StakeInfo(
            amount,
            sHOL,
            stakeAt,
            lockUntil
```

```
        );

+       /// @dev Store the staker to stake ids
+       stakerToStakeIds[receiver].push(stakeId);
+       stakeIdToIndex[stakeId] = stakerToStakeIds[receiver].length - 1;

        /// @dev Mint the sHOL and unstake certificate
        uint256[] memory tokenIds = new uint256[](2);
        tokenIds[0] = sHOL_TOKEN_ID;
        tokenIds[1] = stakeId;
        uint256[] memory amounts = new uint256[](2);
        amounts[0] = sHOL;
        amounts[1] = 1;
        _mintBatch(receiver, tokenIds, amounts, '');

-       /// @dev Store the staker to stake ids
-       stakerToStakeIds[receiver].push(stakeId);
-       stakeIdToIndex[stakeId] = stakerToStakeIds[receiver].length - 1;

        /// @dev Transfer the HOL to the contract
        HOL.safeTransferFrom(msg.sender, address(this), amount);

        emit Staked(msg.sender, receiver, stakeId, amount, sHOL,
    lockUntil);
     }
```

3.10 Logic Error in unstake(0) leads to Valid Stake Deletion

SEVERITY: MEDIUM **STATUS:** Fixed

PATH:

Staking.sol

DESCRIPTION:

unstake(uint256 stakeId) retrieves the index of the stake using stakeIdToIndex[stakeId]. However, for the input stakeId = 0 (which is never a valid stake ID as nextStakeId starts at 1), the mapping returns the default value 0.

```
// Staking.sol
    function unstake(uint256 stakeId, address receiver) public {
        uint256 index = stakeIdToIndex[stakeId]; //@audit Returns 0 for
   stakeId=0
        _unstake(stakeId, index, receiver);
    }

    function _unstake(
        uint256 stakeId,
        uint256 index,
        address receiver
    ) internal {
        StakeInfo memory stakeInfo = stakeIdToStakeInfo[stakeId];
   //@audit Empty for stakeId=0
        require(
            stakeInfo.lockUntil <= block.timestamp,
            "stake is not unlocked"
        ); //@audit 0 <= now, Passes

        /// @dev Burn the sHOL and unstake certificate
        uint256[] memory tokenIds = new uint256[](2);
        tokenIds[0] = sHOL_TOKEN_ID;
        tokenIds[1] = stakeId;
        uint256[] memory amounts = new uint256[](2);
        amounts[0] = stakeInfo.sHOL;
        amounts[1] = 1;
```

```
        _burnBatch(msg.sender, tokenIds, amounts);

        /// @dev clean the stake info and stake list
        _removeStakeInfoFromStaker(msg.sender, stakeId, index); //@audit
    Removes element at index (0) from stakerToStakeIds
        delete stakeIdToStakeInfo[stakeId]; //@audit delete
    stakeIdToStakeInfo[stakeId]
    }
```

IMPACT:

User calls unstake(0): 1. 1 unit of sHOL is burned. 2. The user's actual stake at index 0 (e.g., Stake ID 1) is removed from their stakerToStakeIds list. 3. The lost stake can no longer be unstaked normally because it is no longer in the list, though the NFT for Stake ID 1 still exists. 4. stakeIdToIndex for the removed stake is not cleared correctly because delete stakeIdToIndex[0] is called instead.

RECOMMENDATIONS:

Check stakeInfo.stakeAt != 0.

```
    function unstake(uint256 stakeId, address receiver) public {
+       require(stakeIdToStakeInfo[stakeId].stakeAt != 0, "stake not
    found");
        uint256 index = stakeIdToIndex[stakeId];
        _unstake(stakeId, index, receiver);
    }
```

3.11 Missing Boundary Checks for start

SEVERITY: LOW **STATUS:** Acknowledge

PATH:

`Vesting.sol`

DESCRIPTION:

The Vesting contract performs an initialize operation to set the vesting schedule, where the start parameter should be limited to the current timestamp at the minimum.

```
// Vesting.sol
    function initialize(
        // ...
    ) external initializer {
        /// @dev Set token contract
        HOL = IERC20(tokenContract);

        /// @dev Set vesting schedule
        require(_start <= _cliff, "start must be <= cliff");
        require(_cliff <= _end, "cliff must be <= end");
        require(
            _unlockInterval > 0 || _end == _cliff,
            "unlock interval must be > 0 or end == cliff"
        );
        // ...
    }
```

IMPACT:

When start is misconfigured, it may cause users to claim rewards prematurely.

RECOMMENDATIONS:

Limit the scope of start to ensure it occurs at or after the current time.

```
    function initialize(
```

```
        // ...
    ) external initializer {
        /// @dev Set token contract
        HOL = IERC20(tokenContract);

        /// @dev Set vesting schedule
+       uint256 currentTime = block.timestamp;
+       require(currentTime <= _start, "currentTime must be <= _start");
        require(_start <= _cliff, "start must be <= cliff");
        require(_cliff <= _end, "cliff must be <= end");
        require(
            _unlockInterval > 0 || _end == _cliff,
            "unlock interval must be > 0 or end == cliff"
        );
        // ...
    }
```

3.12 Missing check for unlockInterval

SEVERITY: LOW **STATUS:** Fixed

PATH:

`Vesting.sol`

DESCRIPTION:

The initialize function of the Vesting contract is missing a check for the unlockInterval parameter, which could lead to unlockInterval > end - cliff.

```
// Vesting.sol
    function initialize(
        // ...
    ) external initializer {
        /// @dev Set token contract
        HOL = IERC20(tokenContract);

        /// @dev Set vesting schedule
        require(_start <= _cliff, "start must be <= cliff");
        require(_cliff <= _end, "cliff must be <= end");
        require(
            _unlockInterval > 0 || _end == _cliff,
            "unlock interval must be > 0 or end == cliff"
        );
        // ...
    }
```

IMPACT:

If unlockInterval > end - cliff occurs, the step-by-step unlocking logic will fail.

RECOMMENDATIONS:

```
    function initialize(
        // ...
```

```
    ) external initializer {
        /// @dev Set token contract
        HOL = IERC20(tokenContract);

        /// @dev Set vesting schedule
        require(_start <= _cliff, "start must be <= cliff");
        require(_cliff <= _end, "cliff must be <= end");
        require(
            _unlockInterval > 0 || _end == _cliff,
            "unlock interval must be > 0 or end == cliff"
        );
+       require(_unlockInterval < _end - _cliff);
        // ...
    }
```

3.13 amount of 0 leads to a waste of resources

SEVERITY: LOW **STATUS:** Fixed

PATH:

StakingUSD.sol

DESCRIPTION:

The stake() function stores the tokens transferred by the user into the contract. However, there is no minimum transfer limit, allowing users to pass in amount = 0 to conduct a transaction and create a record.

```
// StakingUSD.sol
function stake(string calldata holmesAI, uint256 amount) external {
    StakeRecord storage record = stakes[msg.sender][holmesAI];

    if (record.updateTime == 0) {
        walletToHolmesAI[msg.sender].push(holmesAI);
        holmesAIToWallet[holmesAI].push(msg.sender);
    }

    record.credit += calculateCredit(
        record.staked,
        block.timestamp - record.updateTime
    );
    record.staked += amount;
    record.updateTime = block.timestamp;

    _mint(msg.sender, amount);
    USD.safeTransferFrom(msg.sender, address(this), amount);

    emit Stake(msg.sender, holmesAI, amount, record.credit);
}
```

IMPACT:

When amount = 0, the transaction has no practical significance; it not only consumes resources but

also wastes gas.

RECOMMENDATIONS:

Verify amount > 0.

```
function stake(string calldata holmesAI, uint256 amount) external {
+    require(amount > 0, 'Invalid stake amount');
     // ...
     emit Stake(msg.sender, holmesAI, amount, record.credit);
}
```

3.14 Incorrect Share Minting Allows Fee-On-Transfer Tokens to Drain Vault Funds

SEVERITY: LOW **STATUS:** Fixed

PATH:

`StakingUSD.sol`

DESCRIPTION:

stake() first updates record.staked, then transfers the amount into the contract. The problem here is that if the token contract charges a transaction fee, the actual amount of tokens transferred into the contract will be less than the amount.

```
// StakingUSD.sol
function stake(string calldata holmesAI, uint256 amount) external {
    // ...
    record.staked += amount;
    record.updateTime = block.timestamp;

    _mint(msg.sender, amount);
    USD.safeTransferFrom(msg.sender, address(this), amount);

    emit Stake(msg.sender, holmesAI, amount, record.credit);
}
```

IMPACT:

When the unstake() operation is performed, the contract transfers record.staked to the user, but the user has not actually deposited enough tokens. This will result in losses for other users and may even deplete the contract's token pool.

RECOMMENDATIONS:

It is necessary to ensure that no additional transaction fees are charged for the token, and only valid tokens are supported.

3.15 Reset Min/MaxLockDuration and add limits

SEVERITY: LOW **STATUS:** Fixed

PATH:

`Staking.sol, Airdrop.sol`

DESCRIPTION:

The setMaxLockDuration and setMinLockDuration methods should be limited to a range when executed.

```
// Staking.sol
function setMaxLockDuration(uint256 _maxLockDuration) public onlyOwner {
    maxLockDuration = _maxLockDuration;
}

function setMinLockDuration(uint256 _minLockDuration) public onlyOwner {
    minLockDuration = _minLockDuration;
}
```

IMPACT:

If min > max, it will affect the execution of other transactions and the operation of the project.

RECOMMENDATIONS:

Limit the new values of min and max. And it satisfies minLockDuration <= PART ONE MIN LOCK DURATION < PART ONE MAX LOCK DURATION <= maxLockDuration.

```
function setMaxLockDuration(uint256 _maxLockDuration) public onlyOwner {
+   require(
+       _maxLockDuration >= 2 * 365,
+       "max lock duration must >= 2*365 days"
+   );
    maxLockDuration = _maxLockDuration;
}
function setMinLockDuration(uint256 _minLockDuration) public onlyOwner {
```

```
+   require(
+       _minLockDuration <= 30,
+       "min lock duration must <= 30 days"
+   );
    minLockDuration = _minLockDuration;
}
```

3.16 Inconsistency between code and comments

SEVERITY: INFO **STATUS:** Fixed

PATH:

StakingUSD.sol

DESCRIPTION:

When using hard-coded settings for the USD contract address, the comment uses the BSC-USD contract, but the code actually passes in an EOA account.

```
/// @notice Reference to the Binance-Peg BSC-USD token contract
/// @notice
    https://bscscan.com/token/0x55d398326f99059ff775485246999027b3197955
IERC20 public constant USD =
    IERC20(0xeaf0e4976f408637a6a2941392d6295F2A0C2252);
```

RECOMMENDATIONS:

It should be modified to the Token contract address required by the project.

3.17 Duplicate checks in update cause redundant traversal

SEVERITY: INFO **STATUS:** Fixed

PATH:

Staking.sol

DESCRIPTION:

The overridden update() function checks if it's a mint/burn operation. However, placing from == address(0) || to == address(0) in a for loop causes an extra, meaningless operation to be executed.

```
// Staking.sol
    function _update(
        address from,
        address to,
        uint256[] memory ids,
        uint256[] memory values
    ) internal override {
        for (uint256 i = 0; i < ids.length; i++) {
            if (from == address(0) || to == address(0)) {
                continue;
            }
            require(ids[i] == sHOL_TOKEN_ID, "only sHOL token is
   transferable");
            require(
                isContractTransferable[to] ||
   isContractTransferable[from],
                "only specified contracts are transferable"
            );
        }
        super._update(from, to, ids, values);
    }
```

IMPACT:

Repeated checks lead to redundant traversal, resulting in wasted gas.

RECOMMENDATIONS:

The loop can be exited after the first check.

```
    function _update(
        address from,
        address to,
        uint256[] memory ids,
        uint256[] memory values
    ) internal override {
        for (uint256 i = 0; i < ids.length; i++) {
            if (from == address(0) || to == address(0)) {
-               continue;
+               break;
            }
            require(ids[i] == sHOL_TOKEN_ID, "only sHOL token is
   transferable");
            require(
                isContractTransferable[to] ||
   isContractTransferable[from],
                "only specified contracts are transferable"
            );
        }
        super._update(from, to, ids, values);
    }
```

3.18 No explicit existence check was performed on the non-existent stakeId

SEVERITY: INFO **STATUS:** Fixed

PATH:

`Staking.sol`

DESCRIPTION:

Both getStake() and uri() return a StakeInfo based on the passed tokenId. If the tokenId does not exist, it will return a default value.

RECOMMENDATIONS:

1. The getStake(uint256 stakeId) method must satisfy the condition that stakeId < nextStakeId && stakeId > 0.
2. The uri(uint256 tokenId) must satisfy the condition that tokenId < nextStakeId.

4. CONCLUSION

In this audit, we thoroughly analyzed **HolmesAI** smart contract implementation. The problems found are described and explained in detail in Section 3. The problems found in the audit have been communicated to the project leader. We therefore consider the audit result to be **PASSED**.

To improve this report, we greatly appreciate any constructive feedbacks or suggestions, on our methodology, audit findings, or potential gaps in scope/coverage.

5. APPENDIX

5.1 Basic Coding Assessment

5.1.1 Apply Verification Control

Description	The security of apply verification
Result	Not found
Severity	CRITICAL

5.1.2 Authorization Access Control

Description	Permission checks for external integral functions
Result	Not found
Severity	CRITICAL

5.1.3 Forged Transfer Vulnerability

Description	Assess whether there is a forged transfer notification vulnerability in the contract
Result	Not found
Severity	CRITICAL

5.1.4 Transaction Rollback Attack

Description	Assess whether there is transaction rollback attack vulnerability in the contract
Result	Not found
Severity	CRITICAL

5.1.5 Transaction Block Stuffing Attack

Description	Assess whether there is transaction blocking attack vulnerability
Result	Not found
Severity	CRITICAL

5.1.6 Soft Fail Attack Assessment

Description	Assess whether there is soft fail attack vulnerability
Result	Not found
Severity	CRITICAL

6. DISCLAIMER

This report is subject to the terms and conditions (including without limitation, description of services, confidentiality, disclaimer and limitation of liability) set forth in the Services Agreement, or the scope of services, and terms and conditions provided to the Company in connection with the Agreement. This report provided in connection with the Services set forth in the Agreement shall be used by the Company only to the extent permitted under the terms and conditions set forth in the Agreement. This report may not be transmitted, disclosed, referred to or relied upon by any person for any purposes without ExVul's prior written consent.

This report is not, nor should be considered, an "endorsement" or "disapproval" of any particular project or team. This report is not, nor should be considered, an indication of the economics or value of any "product" or "asset" created by any team or project that contracts ExVul to perform a security assessment. This report does not provide any warranty or guarantee regarding the absolute bug-free nature of the technology analyzed, nor do they provide any indication of the technologies proprietors, business, business model or legal compliance.

This report should not be used in any way to make decisions around investment or involvement with any particular project. This report in no way provides investment advice, nor should be leveraged as investment advice of any sort. This report represents an extensive assessing process intending to help our customers increase the quality of their code while reducing the high level of risk presented by cryptographic tokens and blockchain technology.

Blockchain technology and cryptographic assets present a high level of ongoing risk. ExVul's position is that each company and individual are responsible for their own due diligence and continuous security. ExVul's goal is to help reduce the attack vectors and the high level of variance associated with utilizing new and consistently changing technologies, and in no way claims any guarantee of security or functionality of the technology we agree to analyze.

7. REFERENCES

[1] MITRE. CWE-191: Integer Underflow (Wrap or Wraparound). https://cwe.mitre.org/data/definitions/191.html.

[2] MITRE. CWE-197: Numeric Truncation Error. https://cwe.mitre.org/data/definitions/197.html.

[3] MITRE. CWE-400: Uncontrolled Resource Consumption. https://cwe.mitre.org/data/definitions/400.html.

[4] MITRE. CWE-440: Expected Behavior Violation. https://cwe.mitre.org/data/definitions/440.html.

[5] MITRE. CWE-684: Protection Mechanism Failure. https://cwe.mitre.org/data/definitions/693.html.

[6] MITRE. CWE CATEGORY: 7PK - Security Features. https://cwe.mitre.org/data/definitions/254.html.

[7] MITRE. CWE CATEGORY: Behavioral Problems. https://cwe.mitre.org/data/definitions/438.html.

[8] MITRE. CWE CATEGORY: Numeric Errors. https://cwe.mitre.org/data/definitions/189.html.

[9] MITRE. CWE CATEGORY: Resource Management Errors. https://cve.mitre.org/data/definitions/399.html.

[10] OWASP. Risk Rating Methodology. https://www.owasp.org/index.php/OWASP_Risk_Rating_Methodology

8. About Exvul Security

Premier Security for the Web3 Ecosystem

ExVul is a premier Web3 security firm committed to forging a secure and trustworthy decentralized ecosystem. Our elite team consists of security veterans from world-leading technology and blockchain security firms, including Huawei, YBB Captical, Qihoo 360, Amber, ByteDance, MoveBit, and PeckShield. Team member Nolan is ranked as a top-40 whitehat on Immunefi and is the platform's sole All-Star in the APAC region.

Our expertise covers the full spectrum of Web3 security. We conduct **meticulous smart contract audits**, having fortified thousands of projects on chains like Evm, Solana, Aptos, Sui etc. Our **Blockchain Protocol Audits** secure the core infrastructure of L1/L2 by uncovering deep-seated vulnerabilities. We also offer **comprehensive wallet audits** to protect user assets and provide **proactive web3 pentest**, enabling partners to neutralize threats before they strike.

Trusted by industry leaders, ExVul is the security partner for **OKX, Bitget, Cobo, Infini, Stacks, Aptos, Sui, CoreDAO, Sei** etc.

EXVUL

Contact

Website
www.exvul.com

Email
contact@exvul.com

Twitter
@EXVULSEC

Github
github.com/EXVUL-Sec

ExVul